

# Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

## Contents

|                                        |           |
|----------------------------------------|-----------|
| <b>1. Introduction</b>                 | <b>1</b>  |
| <b>2. Related Work</b>                 | <b>2</b>  |
| <b>3. Dataset and Data Preparation</b> | <b>2</b>  |
| <b>4. Exploratory Analysis</b>         | <b>4</b>  |
| <b>5. Methodology</b>                  | <b>8</b>  |
| <b>6. Results and Discussion</b>       | <b>9</b>  |
| <b>7. Conclusion and Future Work</b>   | <b>13</b> |
| <b>8. References</b>                   | <b>14</b> |

## 1. Introduction

Forecasting mobile network traffic at fine spatial and temporal resolution matters for practical reasons. A network operator who knows how demand will move in the next few minutes at a given location can allocate resources and manage congestion in ways that hourly or daily aggregates do not support. This project investigates one-step-ahead forecasting of Internet traffic using the Milan telecommunications activity dataset, a two-month record of SMS, call, and Internet usage across roughly 10,000 geographical grid squares at 10-minute intervals [1].

The research question has two parts. The first asks how different sequential models compare for one-step-ahead forecasting. The second asks how their performance varies across geographical areas with different traffic characteristics, and this second part is important because model performance may depend on the characteristics of the area itself rather than being a fixed property of the architecture. To investigate this, three sequential models with different modeling mechanisms were evaluated across areas that differ in traffic volume, volatility, and weekday and weekend behavior. The aim was therefore not only to compare prediction errors, but also to examine whether the relative performance of the models changes across areas and what characteristics might explain those differences.

The methodology is evidence-first throughout. Memory optimization was measured before and

after instead of assumed to work. The three highest-traffic squares were computed from the data instead of guessed. Model selection drew on exploratory findings and prior literature instead of being decided in advance. Every number in this report traces to a specific, logged experiment run on documented hardware.

## 2. Related Work

Mobile traffic forecasting has been studied since large urban telecommunications datasets became available for research. The dataset used here originates from Barlacchi et al. [1], who released spatio-temporally resolved SMS, call, and Internet activity records to support exactly this kind of investigation. Several later studies used this dataset family, or comparable large-scale traces, to test sequential deep learning architectures against classical baselines.

Zhang and Patras [2] proposed a spatio-temporal neural network for long-term mobile traffic forecasting, arguing that purely temporal models miss spatial correlation between nearby cells. This project takes the narrower, purely temporal univariate approach as a starting point, since the assignment scope is one-step-ahead forecasting for a single area, not joint spatial-temporal prediction. Feng et al. [3] introduced DeepTP, an architecture combining spatial dependence with long-period temporal structure, motivated by burstiness, by time-of-day and day-of-week effects, and by spatial dependency from population movement. The day-of-week effect turned out to be directly relevant to the failure analysis in Section 6.

Trinh et al. [4] analyzed mobile traffic using real LTE traces at the level of individual base stations, characterizing daily periodicity and proposing statistical models for single cells instead of city-wide aggregates. The same group later applied an LSTM directly to raw traces for both one-step and longer-horizon prediction [5], which motivated including an LSTM among the three models evaluated here. Wang et al. [6] showed that cellular traffic prediction accuracy, and the choice of model that performs best, vary across geographical areas within a single city, attributing this to differences in land use and population mobility. The present results are consistent with that finding, though the explanation offered in Section 6 rests on traffic statistics actually measured in this project, namely weekday and weekend split and volatility, since no external land-use data was available.

Three points from this literature shaped the methodology. Daily and weekly periodicity is a genuine, exploitable structure in mobile traffic data rather than noise to be removed. Recurrent architectures are an established starting point for raw-trace prediction, which motivated the LSTM. Traffic character and the appropriate model choice vary across areas within one city, which motivated evaluating three areas deliberately chosen to differ rather than tuning on a single representative square. No paper reviewed here compares an LSTM, a temporal convolutional network, and a Transformer under matched one-step-ahead conditions on this dataset, which is the specific gap this project addresses.

## 3. Dataset and Data Preparation

### 3.1 Dataset description

The dataset consists of 62 daily files, `sms-call-internet-mi-YYYY-MM-DD.txt`, obtained through the Harvard Dataverse API (DOI 10.7910/DVN/EGZHFV) and covering 2013-11-01 through 2014-01-01. Each file is tab-separated with no header and eight columns: square ID, an epoch timestamp

in milliseconds, country code, SMS-in, SMS-out, call-in, call-out, and Internet traffic. Total raw size across all 62 files is 19.38 GB, with individual files ranging from 322,874,887 to 376,123,111 bytes. File integrity was checked after download by comparing byte counts against the Dataverse metadata, and one file was inspected directly with `head` and `wc` before any parsing code was written, so the schema was confirmed rather than assumed from the assignment brief.

Inspecting the raw rows revealed a structural detail the brief does not mention. Each square-timestamp combination appears once per country code present in that interval, and most of the eight value columns are blank on any given row. Internet traffic for a square at a given interval is the sum across country codes, not a single stored value, and the blank fields mean zero activity for that country code, not missing data. On the first file inspected, this produced 4,842,625 raw rows against 1,439,982 aggregated rows, a 3.3 times inflation that became the main target for the memory optimization described next, rather than dtype reduction on its own.

## 3.2 Memory management

The compute environment was Google Colab: Python 3.13.15, Linux, 12.67 GB RAM, 2 vCPUs, an Intel Xeon at 2.20 GHz, and no GPU. The raw dataset at 19.38 GB already exceeds available RAM, and pandas' default dtype inference would inflate memory use further, so a full naive load was never attempted.

A per-file benchmark was run before any optimization, using `psutil` to measure process RSS. Loading one day's raw file with default pandas settings raised RSS by 307.0 MB and produced a DataFrame with a resident footprint of 295.6 MB in 8.40 seconds. Reading only the three needed columns with narrower dtypes, then aggregating across country code, reduced the RSS delta to 110.0 MB (64.2 percent lower) and the resident footprint to 22.0 MB (92.6 percent lower), and ran faster overall at 6.64 seconds including the aggregation step. Row count fell from 4,842,625 to 1,439,982, a 70.3 percent reduction, confirming that removing the country-code duplication was the main lever, not dtype tuning by itself.

The same pattern held across all 62 files processed one at a time, with each day's result written to Parquet before the next file was loaded. Process RSS never exceeded 1,451.6 MB for the full run, about 13 times smaller than the raw dataset, and total rows fell from 319,896,289 to 89,245,318, a 72.1 percent reduction consistent with the single-file test. Mean processing time was 19.03 seconds per file, well above the 6.64-second single-file benchmark; the difference is Parquet write latency to Google Drive's mounted filesystem, not the pandas computation, since the earlier benchmark did not write to disk. That write cost turned out to matter later, when a Drive synchronization failure required data recovery mid-project (Section 6.2).

Two smaller points are worth recording. The naive-versus-optimized comparison was not run on a freshly restarted runtime, so the absolute RSS figures are not from a clean cold start, though the relative comparison remains fair since both measurements were taken back to back with explicit garbage collection between them. Separately, aggregated rows on the first file, 1,439,982, fell 18 short of the theoretical maximum of 10,000 squares times 144 intervals, indicating a handful of square-interval combinations with no record at all that day, distinct from the country-code blanks discussed above.

### 3.3 Preprocessing, splitting, and leakage prevention

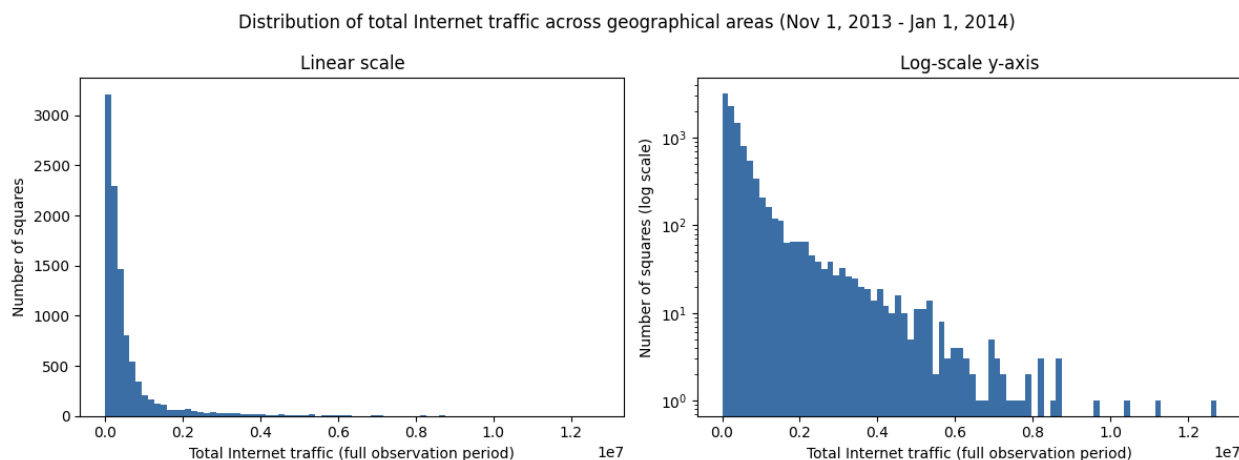
Each square’s series was modeled independently, consistent with the assignment’s univariate, single-area formulation. Splits were chronological with no shuffling: training on 2013-11-01 through 2013-12-08 (38 days), validation on 2013-12-09 through 2013-12-15 (7 days), and test on 2013-12-16 through 2013-12-22 (7 days, the required period). Data after 2013-12-22 was excluded from every split, since it falls after the required test period and using it for training would break chronological order even though it would not leak directly into the December 16-22 predictions.

Scalers were fit on the training split only and applied unchanged to validation and test data. One-step-ahead sequences used the  $L$  most recent observed values ending at time  $t$  to predict the value at  $t+1$ , including, during test evaluation, true historical values from earlier in the test period once the prediction point had moved past them. This is standard one-step-ahead protocol rather than leakage, since the model never sees a target before predicting it and no test-period data informed training or hyperparameter choices.

MAPE excluded points where actual traffic fell below 10.0 traffic units, to avoid division instability, with the excluded count logged alongside every MAPE figure. For the three squares in the final results, no points were excluded in any case, since traffic on these high-volume squares never dropped below the threshold during the test week.

## 4. Exploratory Analysis

### 4.1 Distribution of total traffic across geographical areas



**Figure 1:** Distribution of total Internet traffic across geographical areas, linear and log-scaled y-axis

Total Internet traffic per square over the full period was computed for all 10,000 squares. The distribution is strongly right-skewed (skewness 4.27, mean 555,289 against a median of 277,871), consistent with a small number of high-traffic squares, likely city-center, transit, or commercial areas, alongside many modest-traffic squares. The skew is moderate rather than extreme: the top 1 percent of squares hold only 11.0 percent of total traffic, well short of the 40 to 60 percent that would indicate a handful of squares dominating the city. Coefficient of variation across all squares is about 1.61. For forecasting, this heterogeneity is why models are evaluated on several deliberately different squares here instead of treating one square as representative of the whole city.

## 4.2 Top three areas and required five-square comparison

The three highest-traffic squares, computed from the aggregated data, are 5161 ( $1.274 \times 10^7$ ), 5059 ( $1.117 \times 10^7$ ), and 5259 ( $1.049 \times 10^7$ ). The two additional required squares, 4159 and 4556, rank well outside the top 10, giving useful lower-volume points of comparison.

All five squares have complete data for the first two weeks (2,016 of 2,016 expected intervals, confirmed). The three high-traffic squares are not simply scaled-up versions of the two lower-traffic ones; they differ in relative volatility as well as level. Square 4556 has the lowest coefficient of variation of the five at 0.431, while square 5161, the highest-volume square, has the highest at 0.885. Volume and volatility are measurably different properties here.

Weekday and weekend behavior also diverges, and not always in the direction one might expect. Square 5259 shows a 66.4 percent drop in mean traffic on weekends, the strongest weekday dependence of the five, while square 5161 shows a 26.2 percent weekend increase, the only one of the three high-traffic squares to favor weekends. Square 4556 favors weekends mildly (10.6 percent) and peaks at 22:00, unlike the other four squares, which all peak between 12:00 and 16:00. These are direct observations. One possible explanation, offered as hypothesis since no land-use data was consulted, is that 5259 sits in a business-dominated area while 5161 and 4556 are closer to leisure or residential zones, which would be consistent with Wang et al.'s finding [6] that traffic character tracks area function.

A daily cycle is visible in every square examined, with a trough around 04:00-06:00 and elevated afternoon traffic. This gave initial support for treating roughly 144 ten-minute steps, or 24 hours, as a periodicity worth testing directly, which Section 4.3 does rather than assuming from the plots alone.

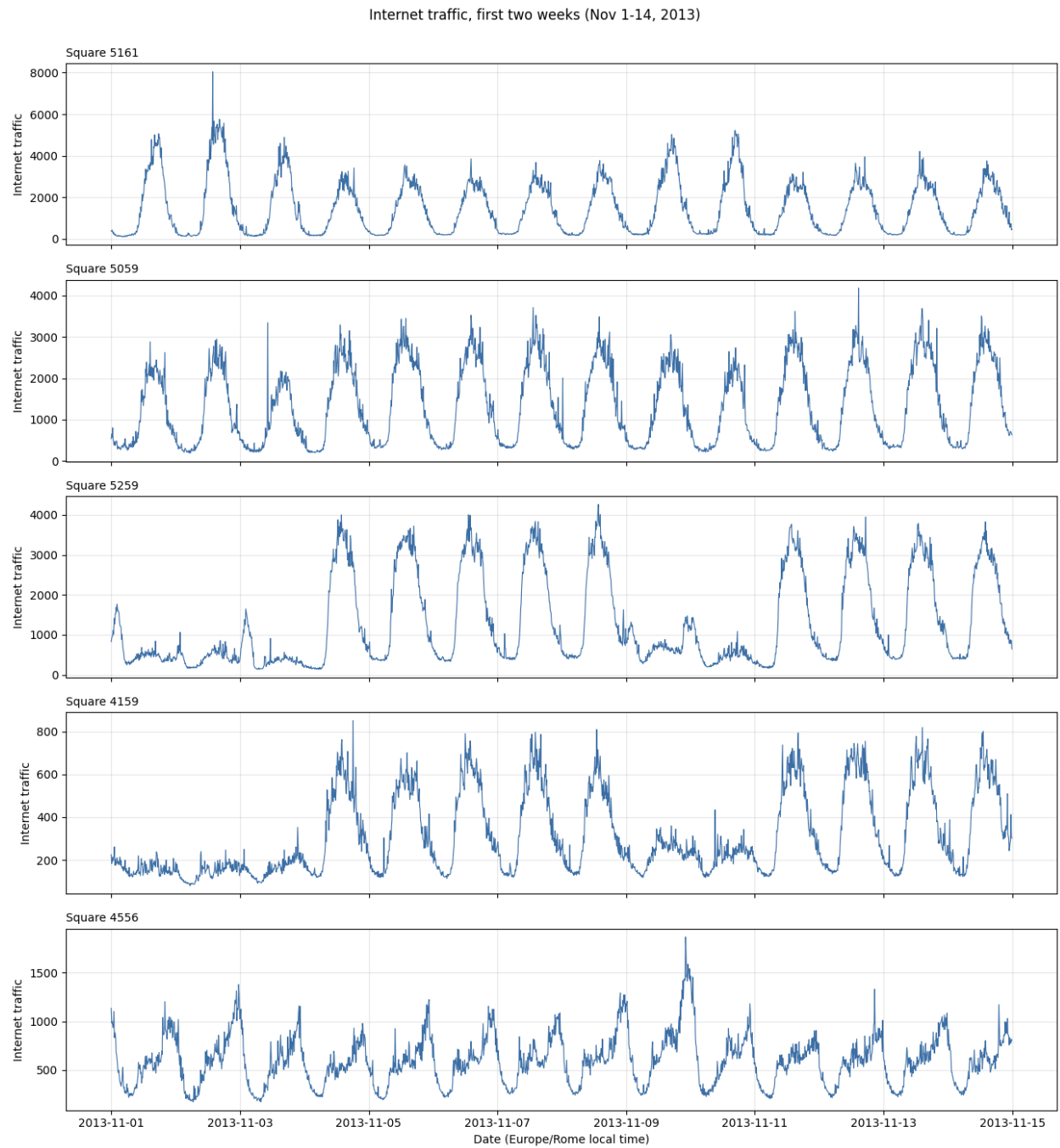
## 4.3 Additional time-series analyses on the highest-traffic square (5161)

Two further analyses were chosen because the findings above raised specific questions, not because they appear on a standard checklist: ACF and PACF, to quantify the periodicity and inform sequence length, and an Augmented Dickey-Fuller test, to check whether differencing is needed.

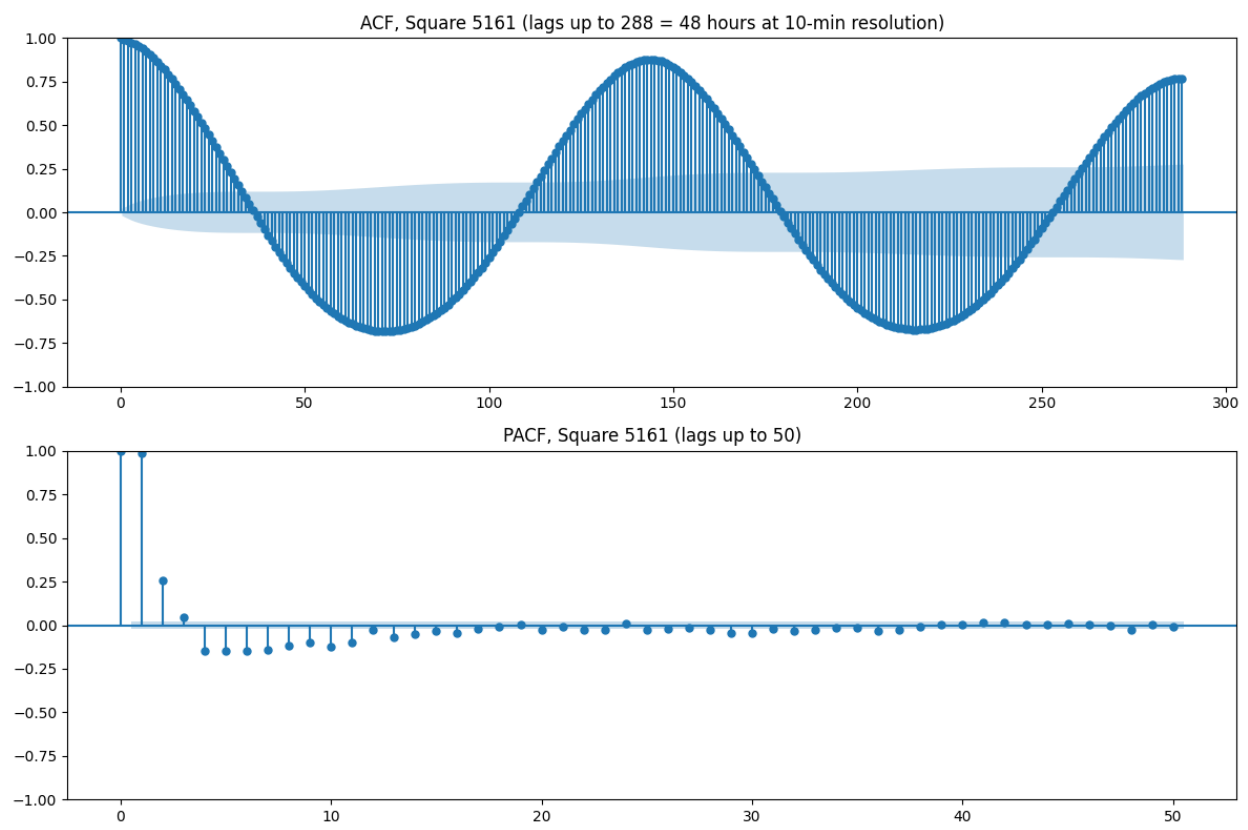
The ADF test on the full 62-day series (8,928 observations, no missing intervals) gives a statistic of -19.03 with p approximately 0.000000, rejecting the unit-root null at the 1 percent level (critical value -3.431). This provides evidence against a unit-root process and suggests differencing is not necessary for removing a stochastic trend. It does not mean the series lacks seasonality; that is a separate question, and the strong daily structure in the ACF below shows the seasonality is real.

ACF at lag 6 (one hour) is 0.939, decaying only to 0.878 at lag 144 (24 hours) and 0.770 at lag 288 (48 hours). High correlation at short lags is unremarkable for a smooth series on its own, so a windowed local-maximum check was used instead of reading the raw values alone: within a 15-lag window around 144, ACF rises to a local peak exactly at lag 144 (0.673, then 0.878, then 0.650 at 129, 144, and 159), and the same pattern holds around lag 288. That local bump, rather than the absolute correlation value, is the evidence for genuine daily periodicity, and it supports dilation schedules and sequence lengths that reach lag 144 in the models below.

PACF is concentrated at lag 1 (0.987) and lag 2 (0.260), with smaller but still significant values out to about lag 11 (threshold approximately 0.0207 at this sample size) before flattening. Recent history dominates direct linear dependence, while the separate daily structure found through ACF operates on a longer horizon that PACF, computed only to lag 20 here, does not directly capture.



**Figure 2:** Internet traffic time series, first two weeks (November 1-14), all five required squares



**Figure 3:** ACF (lags 0-288) and PACF (lags 0-20), square 5161

Together these results set the sequence lengths tested in Section 5: 24 steps as a short baseline past the PACF cutoff, and 144 steps as a deliberate follow-up motivated by the confirmed daily peak.

## 5. Methodology

### 5.1 Forecasting formulation

For area  $a$  and time  $t$ , each model takes the  $L$  most recent observed values ending at  $t$  and predicts the value at  $t+1$ , with  $L$  specified per model below. All three models are trained separately per area rather than jointly, matching the assignment’s single-area formulation.

### 5.2 Baselines

Two baselines were used, chosen for reasons the exploratory analysis already supplied. Persistence, predicting the next value equal to the current one, follows from the near-1.0 lag-1 ACF and PACF. Seasonal-naive, predicting the value 144 steps earlier, follows from the confirmed daily ACF peak. Their role is to show whether the three main models learn anything beyond simple repetition, not to serve as one of the three required architectures.

### 5.3 Model selection

Three architectures were chosen to differ mechanistically rather than as variants of one family.

LSTM was selected as an established approach for raw-trace mobile traffic prediction [4, 5] and fits the strong short-lag PACF dependence found above, since recurrent gating naturally weights recent history heavily while retaining some longer memory.

A temporal convolutional network with dilated causal convolution offers a different mechanism: its dilation schedule can be set to span a target lag directly, here the confirmed 144-step cycle, instead of relying on a recurrent network to discover the periodicity on its own. It is also the cheapest of the three to train on this CPU-only environment.

A Transformer with a self-attention encoder tests a third mechanism: attention can reference any position in the input window directly, without recurrent state or a deep convolutional stack, which might exploit the daily periodicity more directly than either alternative. This was flagged at selection time as the highest computational risk given no GPU, a concern noted here because it shaped how the experiments were planned, even though it did not become a practical obstacle during tuning.

### 5.4 Input representation, training procedure, and iterative tuning

All tuning was done on square 5161 only, changing one variable per experiment and using each result to decide the next. The full 14-experiment log is in `experiments/experiment_log.csv`; the key transitions are summarized here.

The LSTM went through seven experiments. A baseline with sequence length 24, hidden size 32, and one layer beat persistence on MAE and RMSE but not MAPE. Extending sequence length to 144 worsened both MAE and RMSE and roughly doubled training time, showing that this architecture’s recurrent gating gained nothing from the longer window, consistent with PACF’s sharp decay beyond the first several lags; sequence length was reverted to 24. Raising hidden size to 64 gave the largest single improvement, beating persistence on all three metrics at once. Further capacity,

hidden size 128 or two layers at hidden size 64, made both training and validation loss worse and stopped earlier, pointing to an optimization difficulty rather than overfitting, since overfitting would show training loss still improving while validation worsens. A learning-rate check at  $3 \times 10^{-4}$  and  $3 \times 10^{-3}$  against the  $1 \times 10^{-3}$  baseline found the higher rate statistically indistinguishable in accuracy but 45 percent faster, and it was kept for that reason given the CPU-only constraint. Final configuration: sequence length 24, hidden size 64, one layer, no dropout, learning rate  $3 \times 10^{-3}$ , batch size 64, Adam.

The TCN went through four experiments. A four-layer baseline (dilations 1/2/4/8, receptive field 61, sequence length 24, 32 channels) matched the LSTM’s early results. Extending sequence length to 144 and depth to six layers together (receptive field 253) gave the best result seen so far, but changed two things at once. An isolation experiment held sequence length at 144 while reverting to the shallow four-layer network, which does not reach lag 144, and this regressed below even the original baseline, confirming that the earlier gain came specifically from the receptive field reaching the daily periodicity rather than from a longer input alone. Reducing channels from 32 to 16 at the correct receptive field saved 53 percent training time at a cost of about 3.6 percent worse MAE and RMSE, a real trade-off that was not taken given the priority on accuracy. Final configuration: sequence length 144, six layers of 32 channels, kernel size 3, dilations 1/2/4/8/16/32 (receptive field 253), no dropout, learning rate  $1 \times 10^{-3}$ , batch size 64, Adam.

The Transformer went through three experiments. A baseline (sequence length 24, `d_model` 32, two heads, two encoder layers) trained in 28.87 seconds without difficulty, which addressed the computational concern raised at selection time. Extending sequence length to 144 needed no architectural change, since attention can reach lag 144 directly, and improved all three metrics at roughly 4.6 times the training cost, matching the quadratic scaling expected of attention with sequence length. Widening `d_model` from 32 to 64 improved all three metrics again, most notably MAPE, but nearly tripled training time; tuning stopped here given the assignment’s caution against expensive search that is not clearly justified. Final configuration: sequence length 144, `d_model` 64, two heads, two encoder layers, feedforward dimension 64, no dropout, learning rate  $1 \times 10^{-3}$ , batch size 64, Adam.

## 5.5 Final training and evaluation protocol

The first approach to final evaluation retrained each model on combined training and validation data, using a fixed epoch count taken from whichever epoch early stopping had selected during tuning. This produced a badly degraded Transformer on two of three squares, with test MAPE reaching 26.1 percent against a tuning-phase validation MAPE of 7.81 percent. The cause was that an epoch count chosen against one dataset’s loss landscape does not transfer reliably to a differently composed one; the fixed count happened to still work for one square and not the other two. The fix was to train on the original training split only, with validation-based early stopping and best-weight restoration exactly as in tuning, then evaluate once on the untouched test period. This gives up the extra seven days of validation data for final training, a trade-off accepted only after the alternative proved unreliable.

## 6. Results and Discussion

### 6.1 Test-period baseline performance (December 16-22)

| Square | Model          | MAE     | RMSE    | MAPE   |
|--------|----------------|---------|---------|--------|
| 5161   | Persistence    | 92.802  | 134.878 | 9.194  |
| 5161   | Seasonal-naive | 338.594 | 619.040 | 25.940 |
| 5059   | Persistence    | 81.517  | 114.375 | 7.959  |
| 5059   | Seasonal-naive | 171.736 | 245.870 | 18.019 |
| 5259   | Persistence    | 75.968  | 109.578 | 8.115  |
| 5259   | Seasonal-naive | 470.321 | 861.621 | 71.621 |

Seasonal-naive collapses hardest on square 5259 (MAPE 71.6 percent), which is also the square with the sharpest weekday and weekend split found earlier (-66.4 percent). Assuming the same time yesterday predicts today fails worst exactly where day type changes traffic character the most.

## 6.2 Final model performance, three required tables

**Table 1, Square 5161**

| Model                   | MAE    | RMSE    | MAPE   |
|-------------------------|--------|---------|--------|
| LSTM                    | 96.012 | 133.562 | 13.460 |
| TCN                     | 94.754 | 129.163 | 14.058 |
| Transformer             | 94.576 | 128.319 | 14.738 |
| Persistence (reference) | 92.802 | 134.878 | 9.194  |

**Table 2, Square 5059**

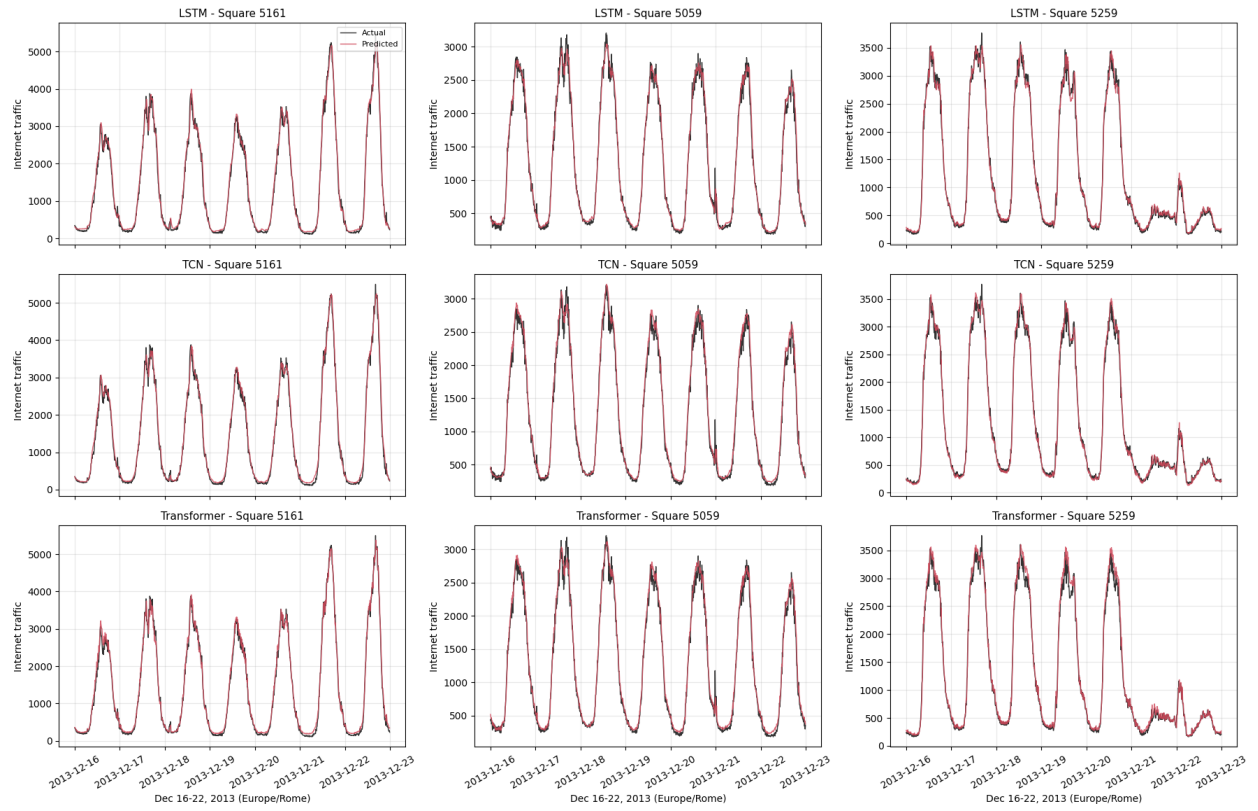
| Model                   | MAE    | RMSE    | MAPE  |
|-------------------------|--------|---------|-------|
| LSTM                    | 71.014 | 101.040 | 7.650 |
| TCN                     | 77.644 | 108.640 | 8.281 |
| Transformer             | 76.572 | 103.926 | 9.022 |
| Persistence (reference) | 81.517 | 114.375 | 7.959 |

**Table 3, Square 5259**

| Model                   | MAE    | RMSE    | MAPE  |
|-------------------------|--------|---------|-------|
| LSTM                    | 71.345 | 99.801  | 8.601 |
| TCN                     | 65.923 | 94.559  | 7.465 |
| Transformer             | 79.022 | 115.892 | 8.840 |
| Persistence (reference) | 75.968 | 109.578 | 8.115 |

Table 1 holds the most striking result in the project. On square 5161, persistence has the lowest MAE of all four entries and, by a wide margin, the lowest MAPE (9.194 against 13.460 for the

Actual vs Predicted Internet Traffic, December 16-22 Evaluation Period



**Figure 4:** Actual versus predicted Internet traffic for all nine model and square combinations, December 16-22 evaluation period

best model). Only the Transformer’s RMSE (128.319) beats persistence, and by a modest amount. None of the three sequential models earns a clear advantage here.

### 6.3 Training and execution time

Timing used a high-resolution performance counter on the documented hardware (Colab, 2 vCPUs, no GPU). Training time covers the full early-stopping loop for the final model on each square; inference time covers one batched forward pass generating all 1,008 test-period predictions.

| Model       | Training time (s), mean across<br>3 squares | Inference time (s),<br>approximate |
|-------------|---------------------------------------------|------------------------------------|
| LSTM        | roughly 25                                  | 0.06-0.19                          |
| TCN         | roughly 190                                 | roughly 0.7                        |
| Transformer | roughly 270                                 | 1.6-2.6                            |

Exact per-run values are in `results/tables/final_test_metrics.csv`. The Transformer costs about ten times what the LSTM does to train, and Table 1 shows that cost does not buy a reliable accuracy advantage on the square where it would matter most.

### 6.4 Comparative analysis across areas

Model ranking is not stable across the three squares, and the pattern lines up with the volatility differences established during exploratory analysis. On square 5059, the steadiest of the three by coefficient of variation (0.716), the LSTM wins every metric and every model beats persistence comfortably. On square 5259, the sharpest weekday and weekend split, the TCN wins every metric, again with every model ahead of persistence. On square 5161, the most volatile square examined in this project (CV 0.885), the pattern breaks: persistence beats all three models on MAE and MAPE, and only the Transformer edges ahead on RMSE.

The result is particularly interesting because the difficulty on 5161 is not limited to one architecture. All three sequential models perform poorly relative to persistence on this square, whereas each of them performs well on at least one of the other two. Read together, the three squares describe a relationship between volatility and model value rather than three unconnected outcomes: added structure, whether recurrent or convolutional, helps clearly on calmer, more periodic squares, but on the single most erratic series in the dataset a simple one-step lookback is already close to the best available answer. This connection was available before any model was trained, since square 5161’s volatility was measured in Section 4.2.

### 6.5 Failure analysis

The Transformer was chosen for closer inspection because its performance swings the most across areas, from a narrow RMSE win on 5161 to being beaten outright by persistence on 5259, rather than because it is the weakest model overall.

The error pattern on square 5259 is clear in the timestep-level results: the ten largest errors are all overpredictions, concentrated between 12:00 and 18:00 on Thursdays and Fridays. Mean absolute error by day of week falls from 108.9 on Thursday to 42.8 on Sunday, tracking the square’s own -66.4 percent weekday-to-weekend traffic gap closely. One possible explanation is that the Transformer’s

attention over a full week of context blends weekday and weekend patterns together. This is only a hypothesis, since the experiment did not include explicit day-type features or an attention analysis that would test the mechanism directly.

On square 5161 the pattern is less orderly. The largest error, an underprediction of 556 units on December 17 at 16:00, occurs during an unexpected weekday spike, a straightforward missed-peak case. The day-of-week breakdown here does not follow the 5259 pattern; the highest mean errors fall on Saturday and Sunday instead of concentrating on a particular weekday, and several other large errors are overpredictions during the square’s own extended afternoon plateau. The absence of a clean pattern is itself a finding, consistent with 5161 being the most volatile series in the project. A model built on average dynamics is likely to be least predictable, not just least accurate, on the series with the least regular behavior, though this stays an interpretation, not something demonstrated directly.

## 7. Conclusion and Future Work

**Answer to the research question.** Model performance for one-step-ahead traffic forecasting is not stable across geographical areas, and the instability tracks a specific, measurable property of each area: volatility. On the two calmer squares evaluated here, one sequential model beat the others clearly and all three beat persistence. On the most volatile square, persistence itself was the strongest single baseline on two of three metrics, and no sequential model reliably justified its added complexity.

**Main findings.** The country-code duplication in the raw data, a factor of 3.3, was the main driver of memory use, ahead of dtype choices. Daily periodicity in Internet traffic is real, confirmed through a windowed ACF check rather than assumed from decay curves. Sequence length affected the LSTM and TCN in opposite directions, hurting the former and helping the latter once its receptive field genuinely reached the daily lag. Which model performed best changed from square to square, and the direction of that change matched each square’s own volatility.

**What the results mean.** The clearest implication is that architectural sophistication does not translate into forecasting value on its own. The Transformer needed no structural change to reach the confirmed daily periodicity, unlike the TCN, and still failed to beat a naive baseline on the square where volatility was highest. Whether a model’s added structure helps appears to depend more on how learnable the underlying signal is than on the model’s capacity or mechanism, a conclusion that only became visible once volatility was measured directly during exploratory analysis rather than left implicit.

**Limitations and future work.** Hyperparameters were tuned on square 5161 only and applied unchanged elsewhere, a choice made under the project’s time and compute constraints that itself produced evidence, since a fixed configuration did not generalize evenly across areas with different traffic character. A reproducibility gap was also found in the final evaluation code: models were constructed before the function’s internal random seed was set, so the first model processed in a session inherited whatever random state preceded it, while later models benefited from an already-reset, deterministic sequence. Comparing two independent runs of the same evaluation reproduced two of three squares exactly and showed real drift on the one processed first; the fix, seeding before constructing each model, is noted for future work rather than applied retroactively here. Other limitations include the roughly two-month observation window, which limits conclusions about longer seasonal effects, the use of only three of 10,000 available squares, and the absence of

exogenous features or spatial data beyond the single Milan grid, despite a separate Milano Grid dataset being available that could help explain why the squares examined here differ as much as they do. Worthwhile next steps, each tied to a specific finding instead of listed generically, include per-area hyperparameter tuning given the demonstrated area-dependence of model ranking, day-of-week features for the Transformer given the weekday-blending pattern found on square 5259, and probabilistic or quantile forecasting for square 5161, where every architecture tested here understated the difficulty of the task.

## 8. References

- [1] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, “A multi-source dataset of urban life in the city of Milan and the Province of Trentino,” *Sci. Data*, vol. 2, no. 150055, 2015.
- [2] C. Zhang and P. Patras, “Long-Term Mobile Traffic Forecasting Using Deep Spatio-Temporal Neural Networks,” in *Proc. 18th ACM Int. Symp. Mobile Ad Hoc Netw. Comput. (MobiHoc)*, Los Angeles, CA, USA, 2018, pp. 231-240.
- [3] J. Feng, X. Chen, R. Gao, M. Zeng, and Y. Li, “DeepTP: An End-to-End Neural Network for Mobile Cellular Traffic Prediction,” *IEEE Netw.*, vol. 32, no. 6, pp. 108-115, 2018.
- [4] H. D. Trinh, N. Bui, J. Widmer, L. Giupponi, and P. Dini, “Analysis and Modeling of Mobile Traffic Using Real Traces,” in *Proc. IEEE 28th Annu. Int. Symp. Personal, Indoor, Mobile Radio Commun. (PIMRC)*, 2017, pp. 1-6.
- [5] H. D. Trinh, L. Giupponi, and P. Dini, “Mobile Traffic Prediction from Raw Data Using LSTM Networks,” in *Proc. IEEE 29th Annu. Int. Symp. Personal, Indoor, Mobile Radio Commun. (PIMRC)*, 2018, pp. 1827-1832.
- [6] X. Wang, Z. Zhou, F. Xiao, K. Xing, Z. Yang, Y. Liu, and C. Peng, “Spatio-Temporal Analysis and Prediction of Cellular Traffic in Metropolis,” *IEEE Trans. Mob. Comput.*, vol. 18, no. 9, pp. 2190-2202, 2019.

GitHub repository: to be added once pushed. Demo video: to be added once recorded.